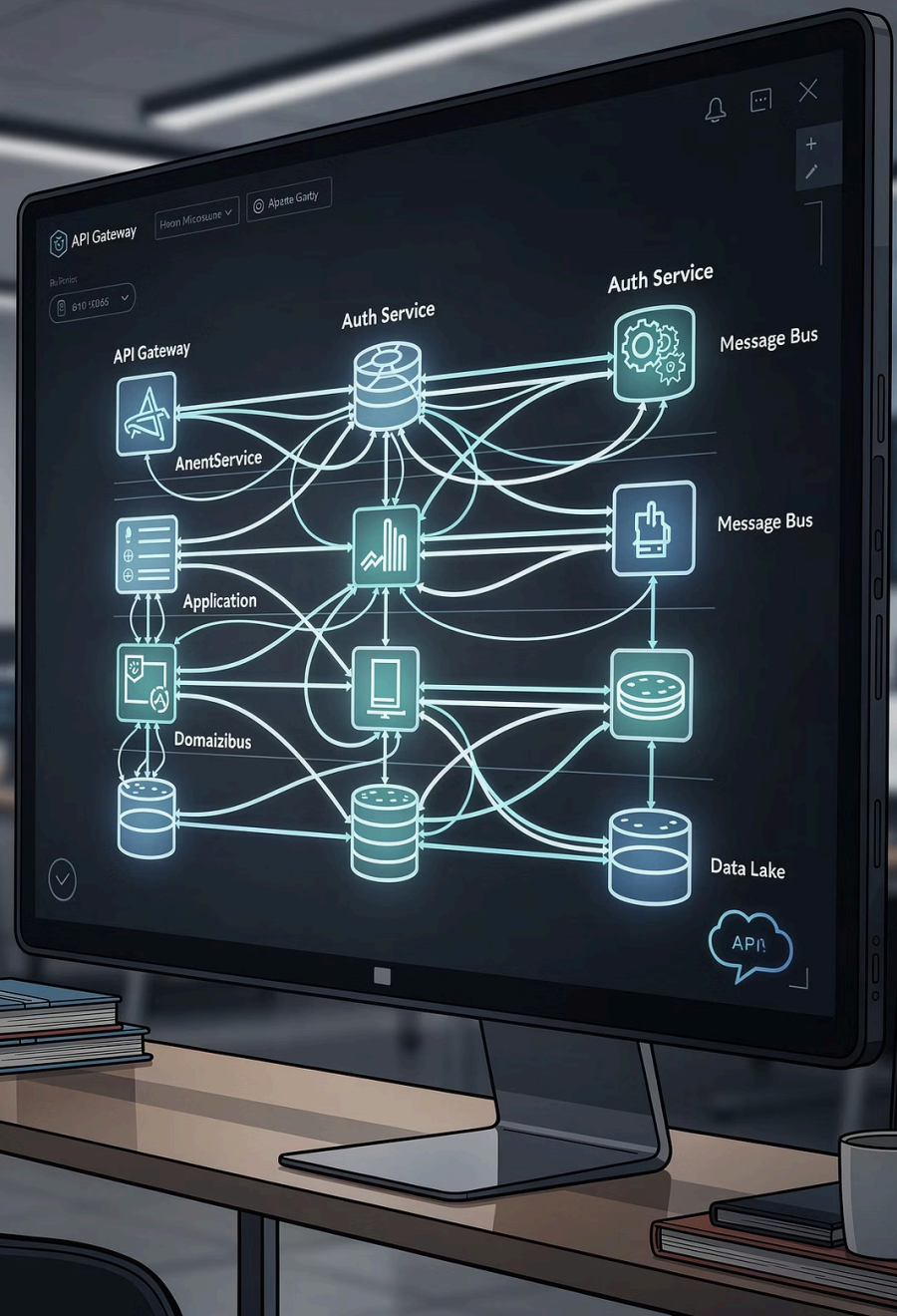




ERP Contable Financiero Enterprise

Documento de Levantamiento Técnico — Arquitectura & Especificación de Sistema

Documento oficial de arquitectura, diseño técnico y especificación de requerimientos de software (SRS/TDD/SAD) para el desarrollo del ERP Contable Financiero. Versión 1.0 — Clasificación: Confidencial — Uso Interno del Equipo de Desarrollo.

SRS

TDD

SAD

DEVOPS

QA

Índice del Documento

Estructura del Levantamiento Técnico

Este documento cubre de forma exhaustiva todos los aspectos técnicos necesarios para implementar el ERP sin necesidad de un nuevo levantamiento funcional. Sirve como referencia única para arquitectos, desarrolladores, QA, DevOps y DBAs.

O1

Fundamentos y Arquitectura

Introducción, objetivos, alcance, arquitectura general, física, lógica, C4 y microservicios.

O3

Datos, APIs y UX

Modelo entidad-relación completo, definición de APIs REST, especificación UX/UI por pantalla.

O2

Dominio, Seguridad y Módulos

Modelo de dominio DDD, seguridad JWT/RBAC, multiempresa, módulos funcionales y reglas de negocio.

O4

DevOps, QA y Continuidad

CI/CD, Docker, monitoreo, auditoría, performance, escalabilidad y recuperación ante desastres.

Capítulo 1 — Visión General

Contexto, Objetivos y Alcance del Proyecto

El proyecto consiste en construir un **ERP Contable Financiero completamente nuevo**, diseñado para operar como plataforma web modular, multiempresa, escalable y preparada para crecer durante los próximos 10 años. La solución reemplaza sistemas legados con una arquitectura desacoplada de alto rendimiento y alta disponibilidad.

Naturaleza

Sistema ERP web, modular, multitenant, con arquitectura desacoplada y orientada a eventos.

Horizonte

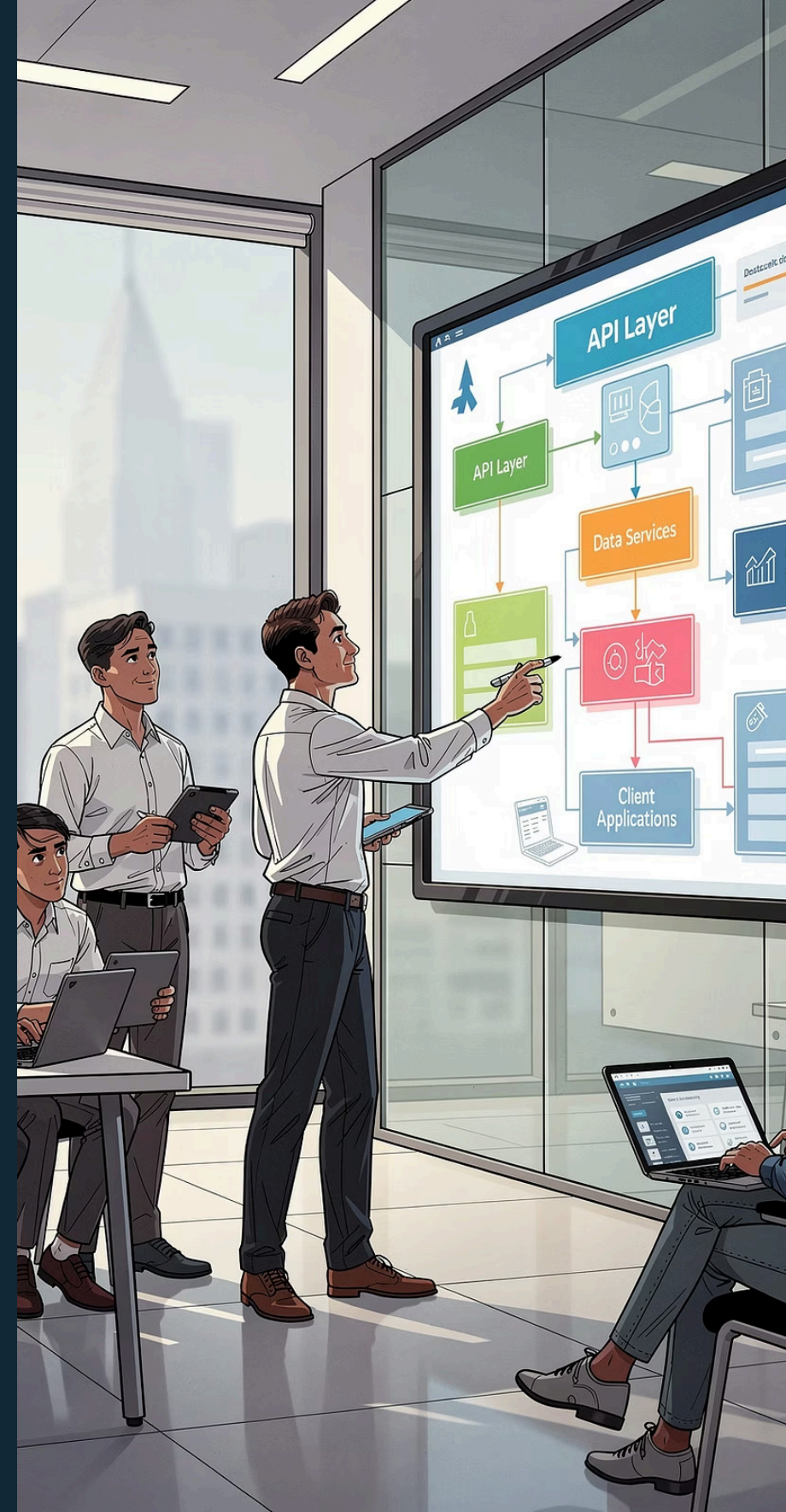
Preparado para soportar crecimiento orgánico durante 10 años sin rediseño estructural.

Audiencia

Arquitectos, Tech Leads, Developers, QA, DevOps, DBA, Product Owner y Cliente.

Clasificación

Documento oficial. Equivalente en calidad a entregables de Accenture, Deloitte, IBM Consulting.



Arquitectura General del Sistema ERP

La arquitectura integra principios de **Clean Architecture**, **Domain Driven Design (DDD)** y **Hexagonal Architecture**, garantizando bajo acoplamiento, alta cohesión y testeabilidad total. El sistema opera como un conjunto de módulos independientes orquestados por un API Gateway central.



Cada módulo implementa su propia lógica de dominio aislada, comunicándose mediante interfaces definidas. La capa de infraestructura es completamente intercambiable sin afectar el dominio de negocio, cumpliendo el principio de inversión de dependencias de SOLID.

Capítulos 4-8 — Stack Tecnológico

Tecnologías y Principios de Diseño

Frontend

- **React + TypeScript** — SPA con tipado estricto
- **Material UI** — Design system empresarial
- **Redux Toolkit** — Estado global predecible
- **React Router** — Navegación declarativa
- **Axios** — Cliente HTTP con interceptores
- Responsive Design + PWA Ready

Backend

- **Node.js + NestJS** — Framework empresarial modular
- **REST API + Swagger** — Contratos documentados
- **JWT + Refresh Token** — Autenticación stateless
- **Repository Pattern + DI** — Desacoplamiento total
- **DTO + Validation Pipes** — Seguridad en entrada

Base de Datos

- **PostgreSQL** — ACID, índices, constraints
- Soft Delete, Auditoría, Versionamiento

Módulos Funcionales del ERP

El ERP se compone de 14 módulos funcionales implementados desde la primera versión. Cada módulo es un dominio acotado (*Bounded Context* en DDD), con su propio modelo de datos, casos de uso, APIs, permisos y auditoría independiente.



Administración & Seguridad

Gestión de usuarios, roles, permisos RBAC, empresas, sucursales.
Autenticación JWT con refresh token y control de sesiones concurrentes.



Contratistas & Insumos

Registro de proveedores, contratos, condiciones comerciales. Catálogo de insumos con unidades de medida, precios y trazabilidad de movimientos.



Comercial & Facturación

Órdenes de venta, cotizaciones, notas de crédito. Integración con GoSocket como proveedor de facturación electrónica (PFE) desacoplado.



Contabilidad & Tesorería

Plan de cuentas, asientos contables, centros de costo, flujo de caja, conciliación bancaria y gestión de pagos.



Presupuestos & Dashboard

Elaboración y seguimiento presupuestario. Dashboard ejecutivo con KPIs en tiempo real, alertas configurables y drill-down por dimensión.



Workflow, Reportes & Auditoría

Motor de aprobaciones configurable, reportes operacionales y financieros con exportación PDF/Excel/CSV, trazabilidad total de operaciones.

Arquitectura de Integración con GoSocket (PFE)

El ERP **no implementa motor propio de facturación electrónica**. Toda la comunicación tributaria es delegada a **GoSocket** como Proveedor de Facturación Electrónica, desacoplado mediante la interfaz `IElectronicInvoiceProvider`. Esto permite cambiar de proveedor en el futuro sin modificar la lógica del ERP.

Responsabilidad del ERP

- Construcción del documento comercial
- Validaciones de negocio y cálculo de impuestos
- Construcción del JSON para GoSocket
- Consumo de API REST y actualización de estados
- Almacenamiento de XML y PDF
- Visualización, reenvíos, logs y auditoría

Responsabilidad de GoSocket

- Firma electrónica y certificados digitales
- Administración de CAF y timbraje
- Generación del XML oficial y envío al SII
- Recepción de respuestas y TrackID
- Acuses de recibo y cumplimiento tributario
- Representación impresa oficial

Seguridad, Multiempresa y Estándares de Desarrollo

Modelo de Seguridad

- **JWT + Refresh Token** rotativo con blacklist
- **RBAC** granular por módulo, acción y empresa
- **HTTPS/SSL** obligatorio en todos los endpoints
- **Fail2Ban** contra ataques de fuerza bruta
- Rate limiting y protección CSRF/XSS/SQLi

Multiempresa

- Aislamiento por **tenant_id** en todas las tablas
- Contexto de empresa inyectado vía JWT claim
- Datos completamente segregados por empresa

Estándares de Desarrollo

- **Clean Architecture**: separación estricta de capas
- **SOLID**: aplicado en cada componente del sistema
- **DRY / KISS**: evitar duplicación y sobrecomplejidad
- **Repository Pattern** para toda la capa de datos
- **DTO + Validation Pipes** en todos los endpoints
- **Swagger** generado automáticamente desde decoradores

GitFlow

- Ramas: `main`, `develop`, `feature/*`, `hotfix/*`
- Pull Requests obligatorios con code review
- Tags semánticos por cada release productivo

DevOps, CI/CD, Monitoreo y Continuidad Operacional

La plataforma se despliega sobre **Ubuntu Server** con **Docker** y **Docker Compose**. El pipeline de CI/CD automatiza compilación, pruebas, análisis estático, construcción de imágenes y despliegue, garantizando releases predecibles y reversibles.

1

Source

Push a GitHub. PR aprobado. Rama `develop` o `main`.

2

Build & Test

GitHub Actions ejecuta lint, unit tests, integration tests y análisis SAST.

3

Containerize

Docker build, tag semántico y push a registry privado con escaneo de vulnerabilidades.

4

Deploy

Docker Compose pull + up en servidor. NGINX recarga configuración. Rollback automático si health check falla.

Monitoreo

Health checks, métricas de servicio, alertas automáticas por umbral.

Logging

Logs estructurados JSON, centralizados por nivel y correlación de requests.

Auditoría

Registro inmutable de cada operación: quién, qué, cuándo, IP y estado.

DR / BCP

Backups automáticos, RTO y RPO definidos, procedimiento de restore documentado.

Roadmap Técnico y Próximos Pasos

El roadmap técnico estructura la entrega en fases iterativas, priorizando la base arquitectónica sobre la que se construirán los módulos de mayor complejidad funcional. Cada fase produce entregables funcionales verificables por QA y el cliente.



❏ Este documento constituye la especificación técnica oficial del proyecto. Ningún requerimiento funcional fuera de este alcance debe desarrollarse sin actualización formal del presente documento.